Prosthetic hand

Table of Contents

1. CHAPTER 1 : PROJECT SPECIFICATIONS 1
2. CHAPTER 2 : DESIGN 3
2.1. Black Box 3
2.2. Control and Execution Unit 3
2.2.1. Mapping the inputs and outputs of the big box on the two components UC and UE. 4
2.2.2. Establishing the resources (EU) 4
2.2.3. Block Diagram for first breakdown 10
2.3. State diagram of Control Unit 11
2.4. Detailed diagram 17
3. CHAPTER 3 : USER MANUAL 19
4. CHAPTER 4 : DESIGN SOLUTIONS JUSTIFICATION 19
5. CHAPTER 5 : FURTHER DEVELOPMENT 19
6. CHAPTER 6 : BIBLIOGRAPHY 20

1. CHAPTER 1 : PROJECT SPECIFICATIONS

You are part of a team tasked with designing a compact and low-cost prosthetic hand, specifically intended for the the amputee that can't afford one. The prosthetic hand should have the basic functionality of a hand like closing when an object is detected and control the hand by using buttons

Operating Procedure:

- **Idle State:**In this state the user can select using the "settingSwitch" switch between settings mode and operating mode.

- **Settings mode:** If the user has selected the setting mode, he can choose using 2 switches, "gestSwitch" and "TiAcSwitch", the setting he wants to change. Using 5 switches the user can create a number in binary system that is introduced in the system if the button "setBtn" is pressed.

- **Operating mode:** The user can choose between 2 modes using the switch "switchMode". Those modes are manual and automatic.
- **Manual mode:** If button "btnClose" is pressed the hand operated by serovs, MG90s, will close. The effect will also be visible on the specific leds ("servoLed1-5" and "modeLed1-2"). If the button "btnOpen" is pressed the hand will open and if the button "btnGesture" is pressed the hand will perform a gesture saved.

- **Automatic mode:** If a object is detected, meaning the “objectDetectBtn” is pressed, it waits 2 seconds, then it retests if the object is really in front of the hand. Once the object is confirmed the hand will be closed for a saved period of time, by default 5 seconds, but if a movement is detected by an accelerometer, ADXL362, the counter will reset and the hand will continue to be go closed. If the time expires, the hand opens.

Safety Feature:

When the hand closes, opens or performs a gesture the hand will close in such a way that the fingers don’t get tangled.

It will be considered that for closing the hand, the index, middle, ring, and little fingers will close before the thumb, with a delay of approximately 1 second. This ensures that the thumb's closing does not interfere with the closure of the other fingers or that the thumb does not close before the others. Similarly, for opening the hand, the thumb will open first, and then after 1 second, the rest of the fingers will open.

For gestures, similar rules will be implemented. Thus, we have:

We will consider the index and middle fingers as **category 1**, and the remaining fingers as **category 2**.

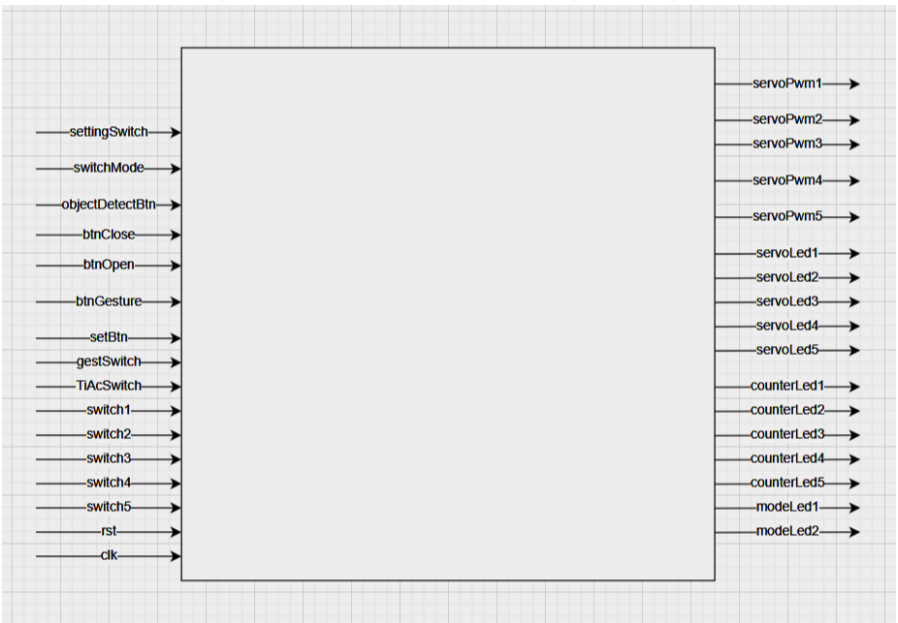
- If we want to close a category 1 finger and the thumb in a gesture, the category 1 finger(s) will close first, and then the thumb will close after 1 second.

- If we want to open a category 1 finger and the thumb simultaneously, the thumb will open first, and then after 0.5 seconds, the rest will open.

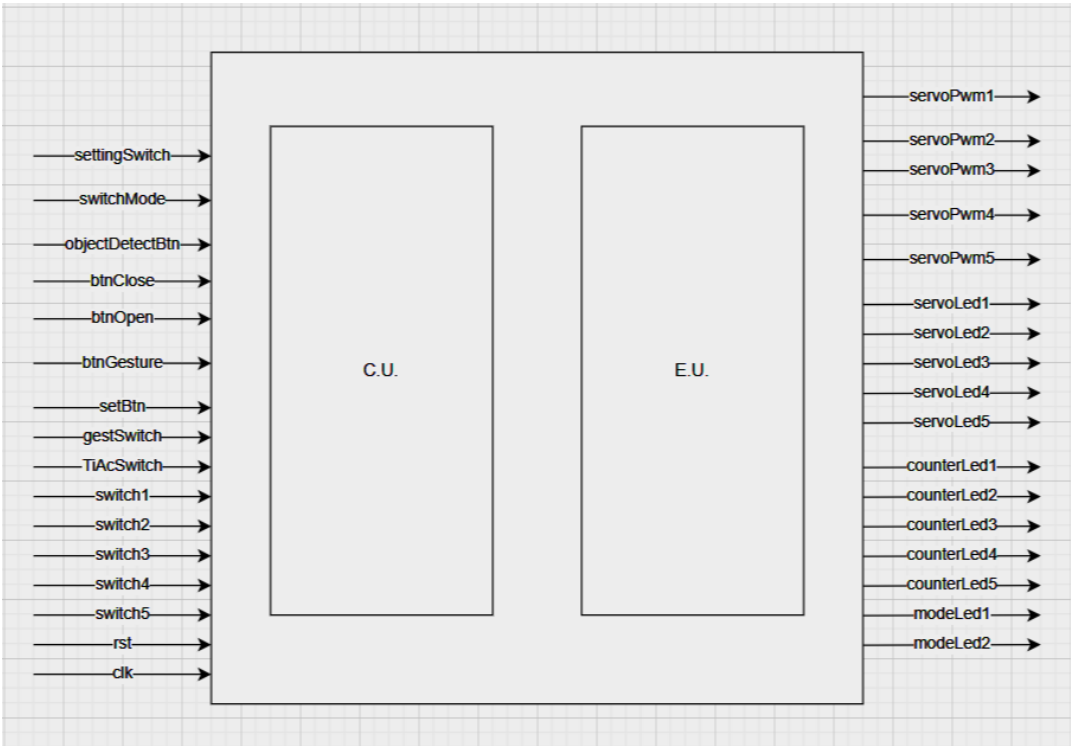
- If we want to open or close category 2 fingers together with the thumb, the fingers will open or close **simultaneously**.

2. CHAPTER 2 : DESIGN

2.1. Black Box



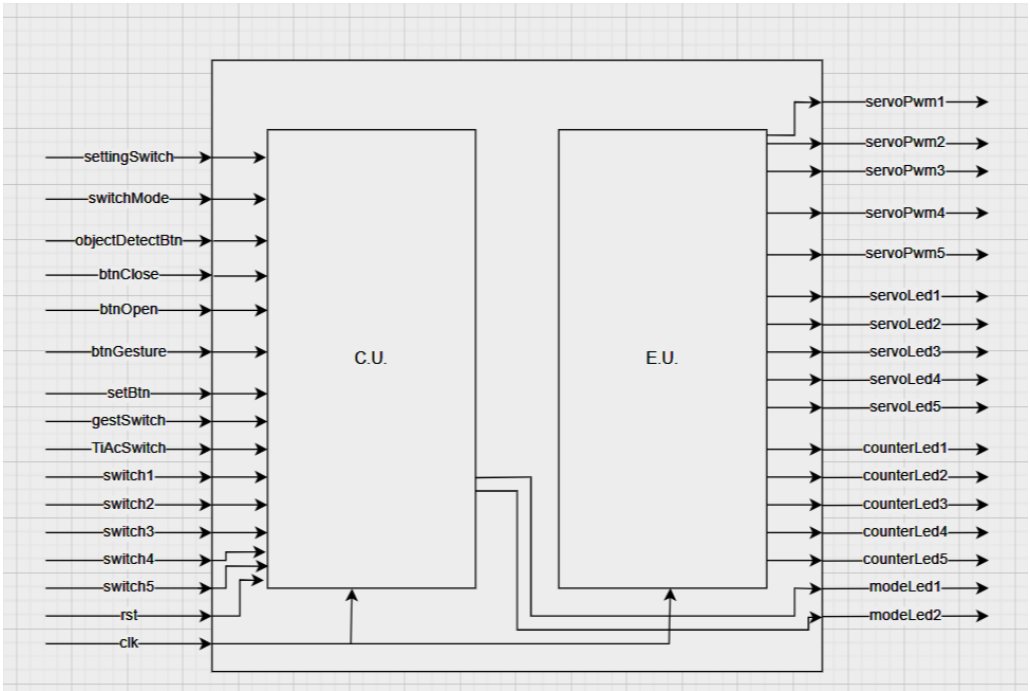
2.2. Control and Execution Unit



The system's black box must be further decomposed in order to find implementable components. We will do a **top-down** breakdown of the problem until we get to known circuits, and then we will implement **bottom-up**.

The first breakdown of any system is one in which we will differentiate between the **control logic** in the system and the **system resources**. The control logic is represented by the Control Unit (CU) and the resources are represented by the Execution Unit (EU).

2.2.1. Mapping the inputs and outputs of the big box on the two components UC and UE.



We can divide both inputs and outputs into 2 categories: **data** and **control**. This separation is essential at the beginning.
Data inputs: the 5 bit number introduced with the help of the 5 switches: switch1, switch2, switch3, switch4, switch5
Control inputs: setingSwitch, switchMode, objectDetectBtn, btnClose, btnOpen, btnGesture, setBtn, gesSwitch, TiAc switch, rst, clk

Data outputs: servoPwm1, servoPwm2, servoPwm3, servoPwm4, servoPwm5, servoLed1, servoLed2, servoLed3, servoLed4, servoLed5, counterLed1, counterLed1, counterLed2, counterLed3, counterLed4, counterLed5, modeLed1, modeLed2
Control outputs: -

2.2.2. Establishing the resources (EU)

In order to further establish the links between the CU and the EU, we must first identify **the resources on the basis of which we make decisions**. These resources can **generate signals to the control unit** and can be **controlled by the CU** via Enable, Reset or other signals.

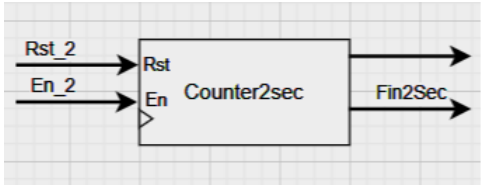
Any decision-making information must come from a resource that generates that information and passes it on to UC.

Resources can be **simple circuits**, which can be implemented directly (counter, register, etc.) or **complex resources** (remainder algorithm, multiplication algorithm, etc.). These complex resources may appear in the first decomposition with black boxes to which we must establish inputs and outputs, but later they must be further decomposed (usually also in CU and EU) until we reach known circuits.

RESOURCES

1. <u>2 seconds Counter2sec counter</u>

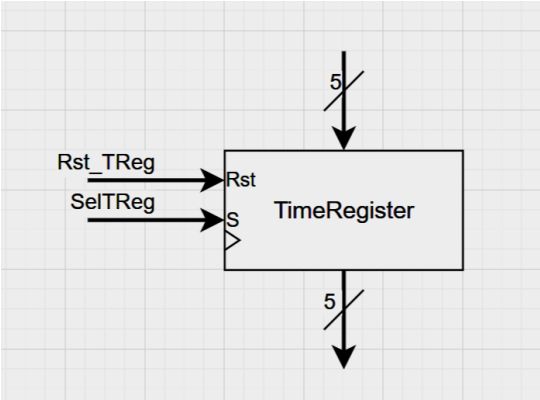
   This is a 2-bit modulo 2 counter. This circuit will have a *Fin2sec* output that will be true when 2 seconds have passed. In addition, we want to start or stop the counter at the right time so it needs an Enable input *En_2* and also a reset *Rst_2*.



   So we have an output *Fin2Sec* signal generated by EU for CU and an *En_2* signal generated by CU to EU. We also have a a*synchronous Rst_2* signal generated by CU.

2. <u>5 bit TimeRegister register</u>

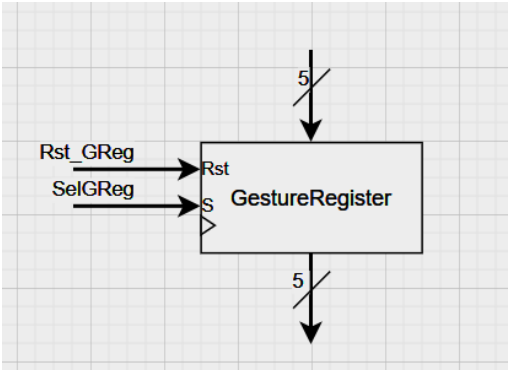
   This is a 5-bit register. This circuit will have a 5 bit input *TimeABCDE* used to parallel load and a 5 bit output *TimeQABCDE*. In addition, we want to reset, load or hold the register at the right time so it needs an Rst_TReg input and also a SelTReg.



   So we have an input *TimeQABCDE* signal generated by EU for Counter5bit also in EU, an *rst_TReg* signal generated by CU to EU, and a SelTReg generated by CU to EU. The register will hold the value 5 by default.

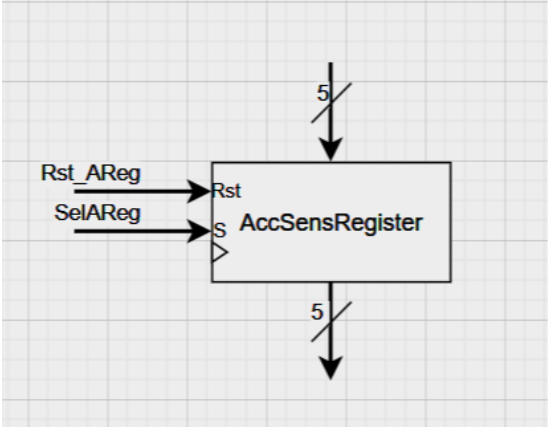
3. <u>5 bit GestureRegister register</u>

   This is a 5-bit register. This circuit will have a 5 bit input *GestABCDE* used to parallel load and a 5 bit output *GestQABCDE*. In addition, we want to reset, load or hold the register at the right time so it needs an Rst_GReg input and also a SelGReg.



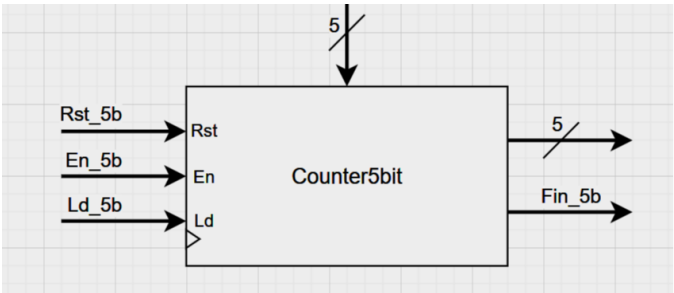
So we have an input *GestQABCDE* signal generated by EU for CU, an *rst_GReg* signal generated by CU to EU, and a SelGReg generated by CU to EU. The register will hold the value “11000” by default.

4. <u>5 bit AccSensRegister register</u>
   This is a 5-bit register. This circuit will have a 5 bit input *AccSensABCDE* used to parallel load and a 5 bit output *AccSensQABCDE*. In addition, we want to reset, load or hold the register at the right time so it needs an Rst_AReg input and also a SelAReg.



So we have an input *SensAccQABCDE* signal generated by EU for CU, an *rst_AReg* signal generated by CU to EU, and a SelAReg generated by CU to EU. The register will hold the value 5 by default.

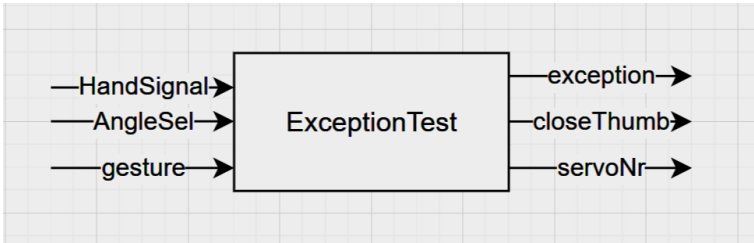
5. <u>X seconds Counter5bit counter</u>
   This is a 5-bit modulo x counter. This circuit will have a *Fin_5b* output that will be true when x seconds have passed. In addition, we want to start or stop the counter at the right time so it needs an Enable input *En_2*, you can also load a number through *dataIn on 5 bits* so it has a *Ld_5b* and also a reset *Rst_5b*.



So we have an input *Fin_5b* signal generated by EU for CU, an *En_5b* signal generated by CU to EU, an *Ld_5b* signal generated by CU to EU, and an dataIn 5-bit signal generated by TimeRegister in EU also for EU. We also have a a*synchronous Rst_5b* signal generated by CU.

6. <u>Combinational circuit - ExceptionTest</u>

This is a combinational circuit helping to determine whether the thumb is going to close or open, if it an exception and all fingers will perform the gesture instantaneously and will provide the way the servos are going to close or open. As input it has HandSignal, AngleSel and gesture and as output it has exception, closeThumb, servoNr.

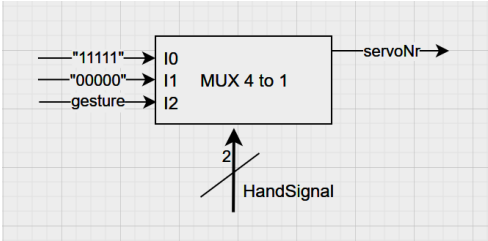


After minimization the functions for the outputs look like this:

-exception = (HandSignal = "10") AND (gesture(3) XNOR AngleSel(3)) AND (gesture(2) XNOR AngleSel(2))

-closeThumb = (HandSignal = "10" AND gesture(4) = '0' AND AngleSel(4) = '1') OR (HandSignal = "01")

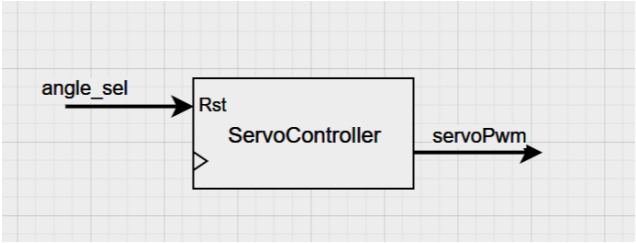
And a simple 4 to 1 MUX on 5 bits for servoNr:



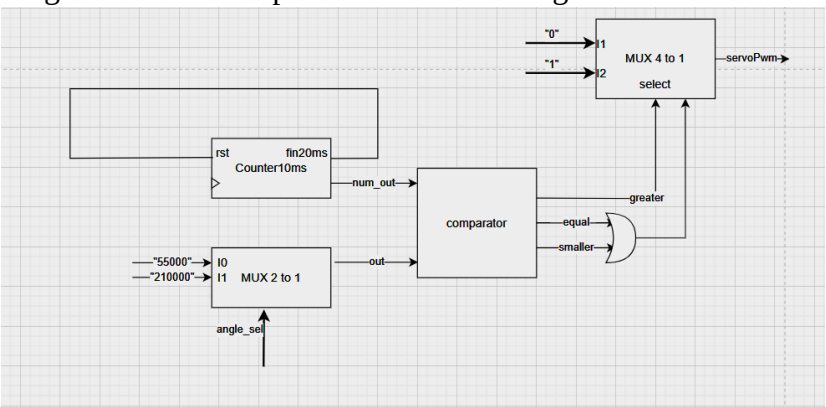
So we have an input *HandSignal* signal generated by CU for EU, an *AngleSel* signal generated by CU to EU, an *gesture* signal generated by gestureRegister, also in EU to EU, and angleSel representing the current status of the servos.

7. <u>SLC circuit - servoController</u>

This is a SLC circuit meant to output the correct pwm in order to operate correctly the servos. It has an input angle_sel to choose the rotation of the servo and an output servoPwm.



The logic diagram for this component is the following:



So we have an input angle_sel signal generated by CU2 for EU and an an servoPWM which is an output.

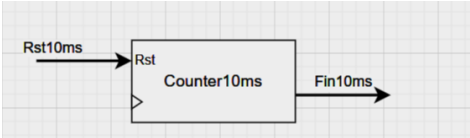
8. <u>0.5 seconds Counter05sec counter</u>

   This circuit will have a *fin05sec* output that will be true when 0.5 seconds have passed. In addition, it also has a reset *Rst_05*.

   So we have an output *Fin05Sec* signal generated by EU for CU. We also have a *synchronous Rst_05* signal generated by CU2.

9. <u>10 ms Counter10ms counter</u>

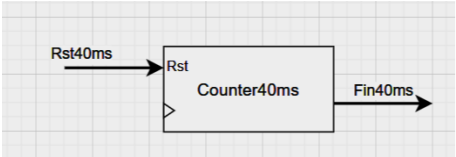
   This circuit will have a *fin10ms* output that will be true when 10 ms have passed. In addition, it also has a reset *Rst10ms*.



   So we have an output *Fin10ms* signal generated by EU for Adxl362_driver. We also have a *synchronous Rst_10ms* signal generated by Adxl362_driver.

10. <u>40 ms Counter40ms counter</u>

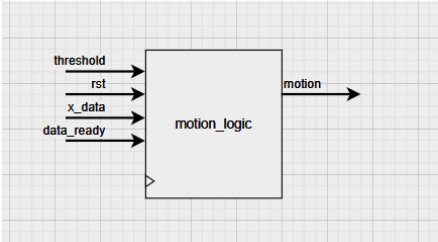
    This circuit will have a *fin40ms* output that will be true when 40 ms have passed. In addition, it also has a reset *Rst40ms.*



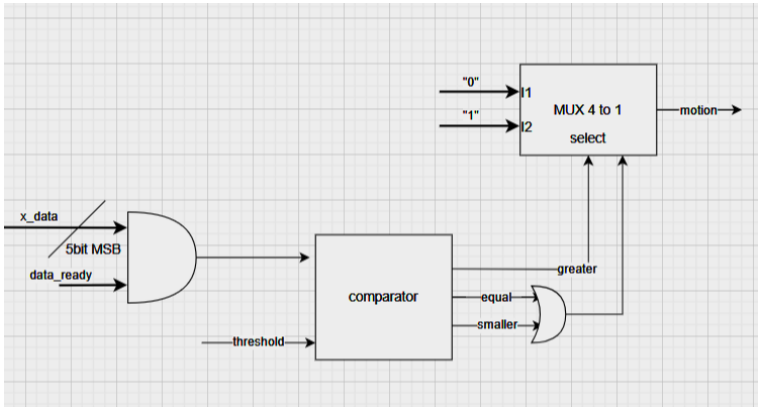
    So we have an output *Fin40ms* signal generated by EU for Adxl362_driver. We also have a *synchronous Rst_40ms* signal generated by Adxl362_driver.

11. <u>SLC circuit – motion_logic</u>

    This is a SLC circuit meant to output correctly if there has been movement detected or not that only updates when the data is correctly read.



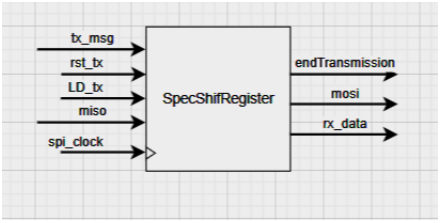
The logic diagram for this component is the following:



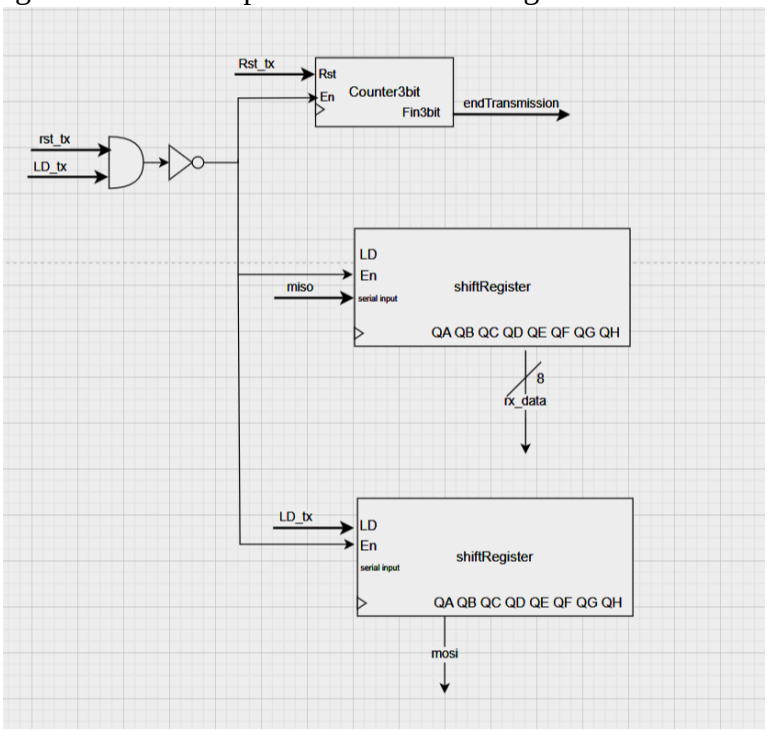
So we have an input threshold signal generated by AccSensRegister, also in EU for EU, an rst generated by CU for EU, an x_data generated by the SpecShiftRegister, also in EU for EU and as output motion generated for CU.

12. <u>SLC circuit – SpecShiftRegister</u>

This is a SLC circuit meant to serially transmit information to the spi device, ADXL362 and serially read from it.

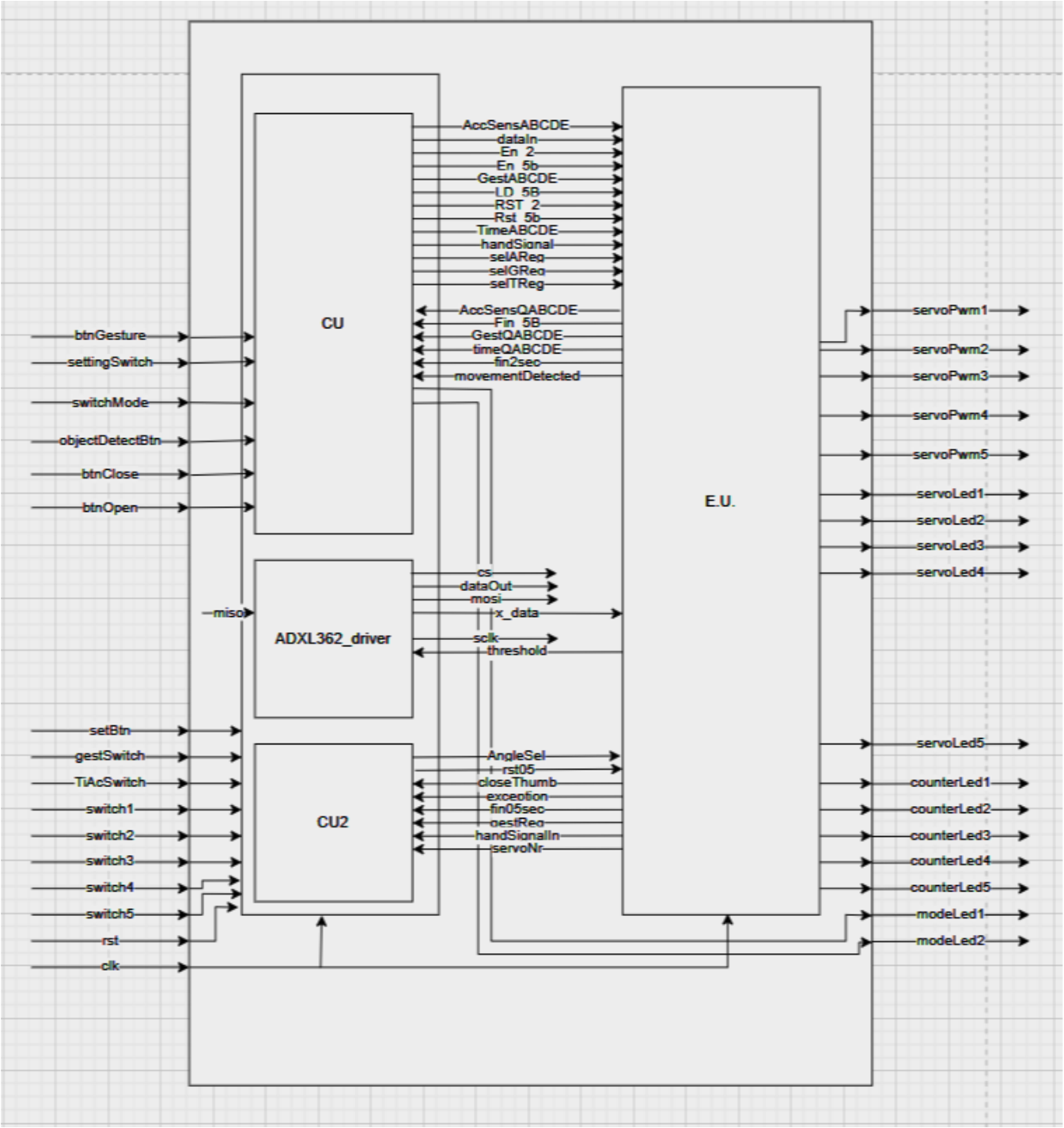


The logic diagram for this component is the following:



So we have an input signal tx_msg generated by adxl362_driver for EU, an input signal rst_tx generated by adxl362_driver for EU, an input signal LD_tx generated by adxl362_driver for EU, an input signal MISO generated by the spi device for the EU, and an spi_clock generated by a clock divider in order to transmit data at 1MHz.

2.2.3. Block Diagram for first breakdown



It is worth noting that CLK has a higher input frequency, so there should be a frequency divider within the EU as another resource to generate the 1 sec period for the 2 counters and another frequency divider for the spi_clk, 1 Mhz.

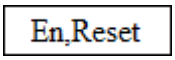
2.3. State diagram of Control Unit

The state diagram is **not a flow chart**, but represents the control part, the decision part of any algorithm, and it can then be implemented directly in VHDL if done correctly.

States are represented by

. A state represents a moment of time (a period).

The **decisions** made in each state are represented by

The **outputs** generated in each state are represented by 

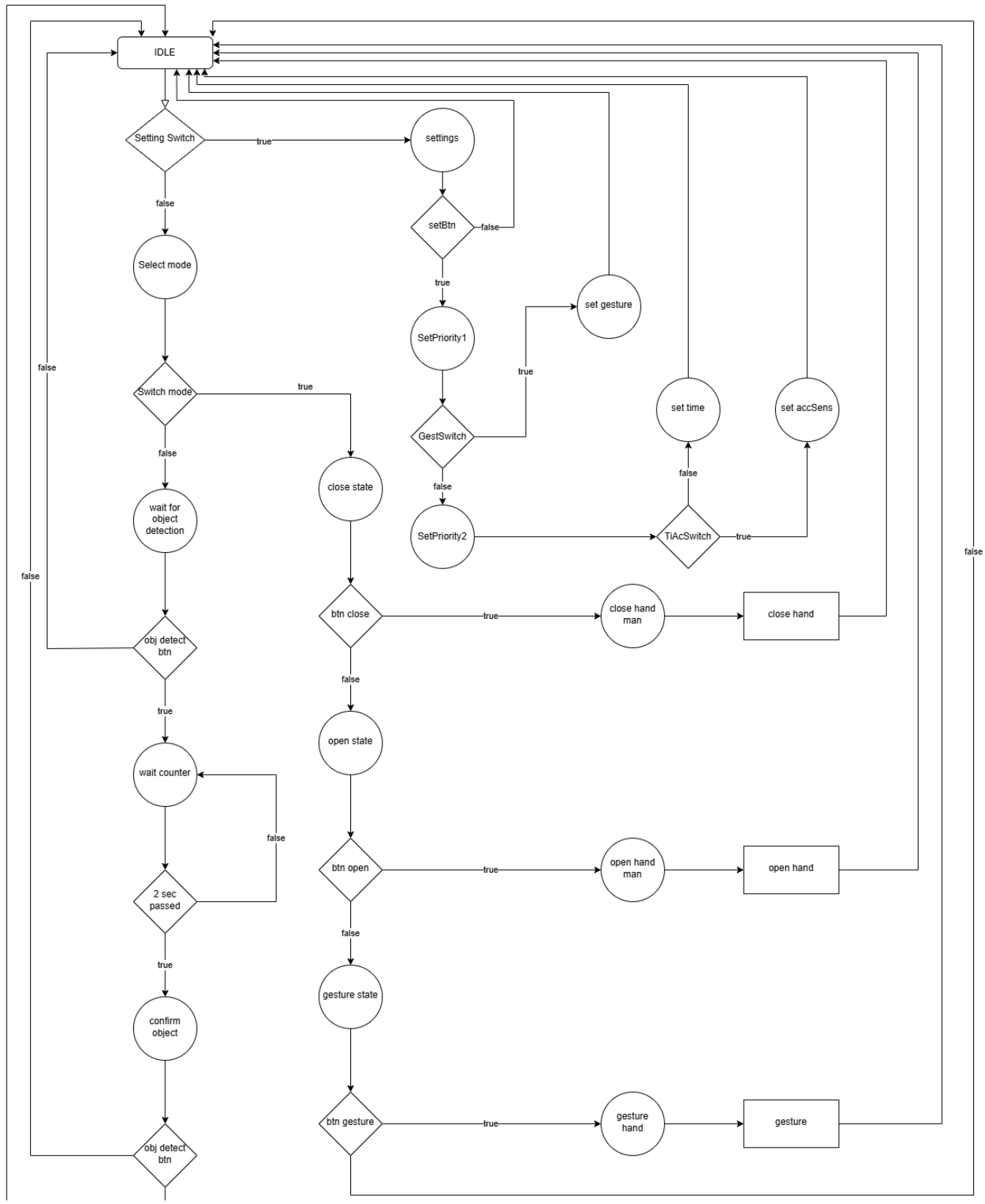
. Inside the rectangle are the outputs that are true at that time.

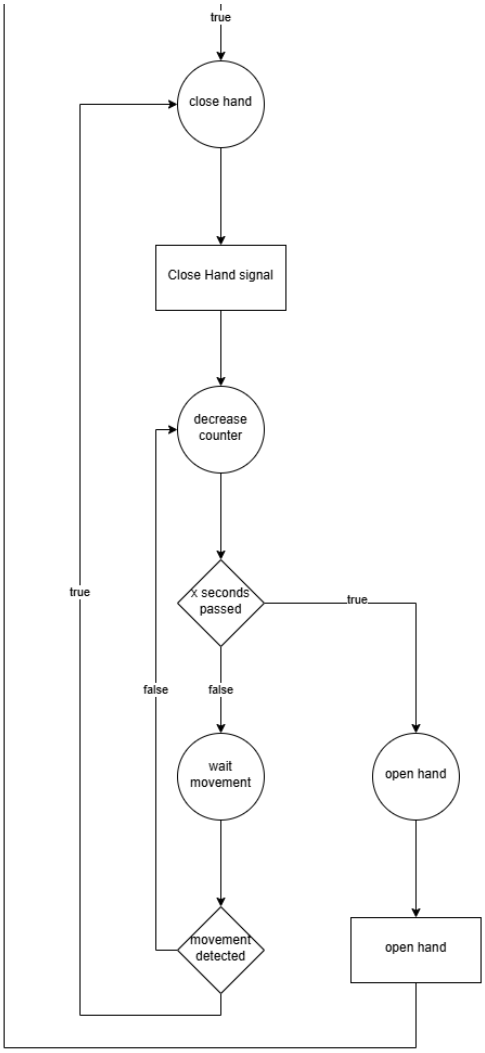
These outputs can be positioned *before* or *after* the decision.

If they are *before* the decision, it means that the outputs depend only on the internal state, so we have a **Moore** automaton.

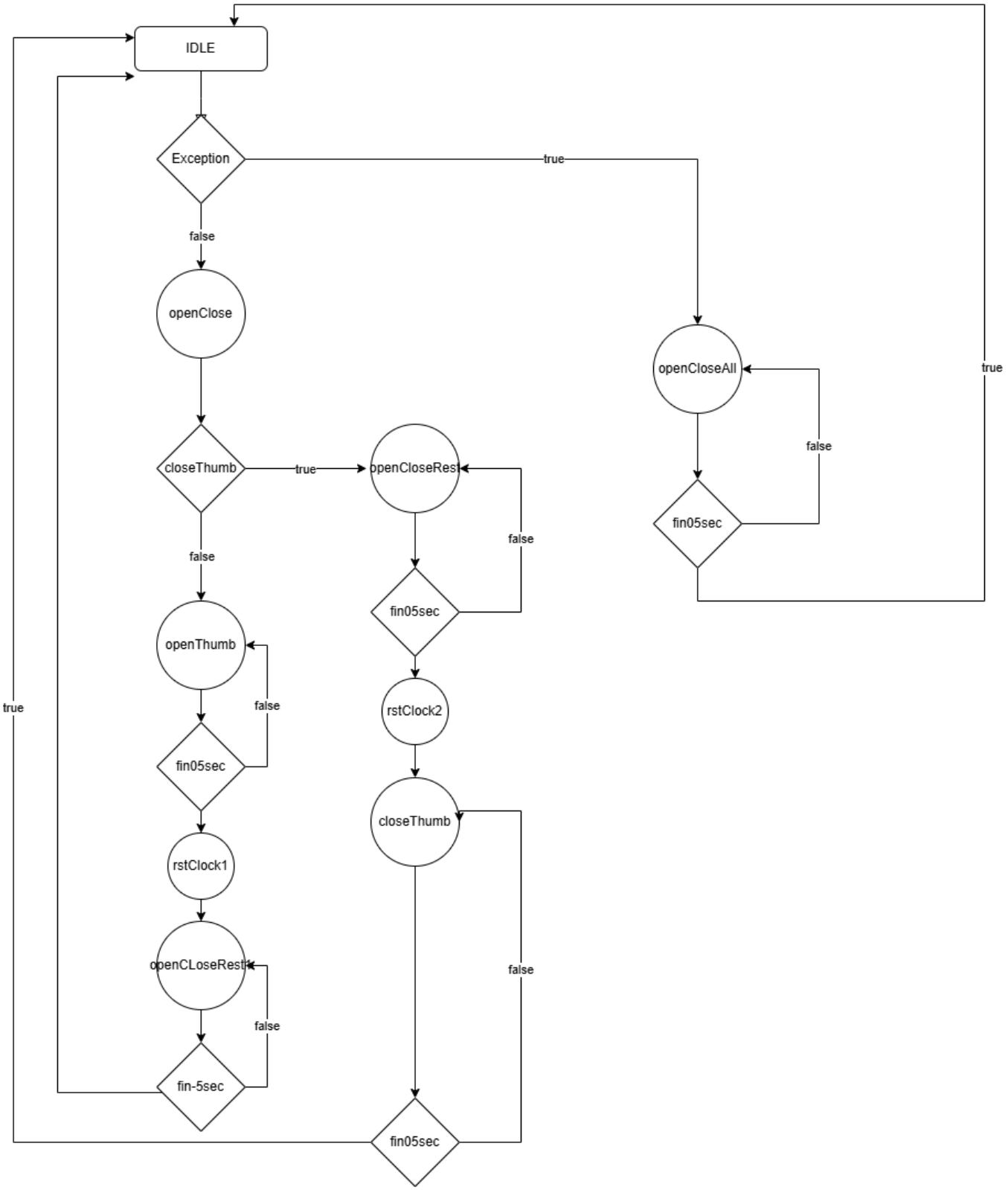
If they are *after* the decision, it means that the output also depends on the internal state and the inputs, so we have a **Mealy** automaton.

CU

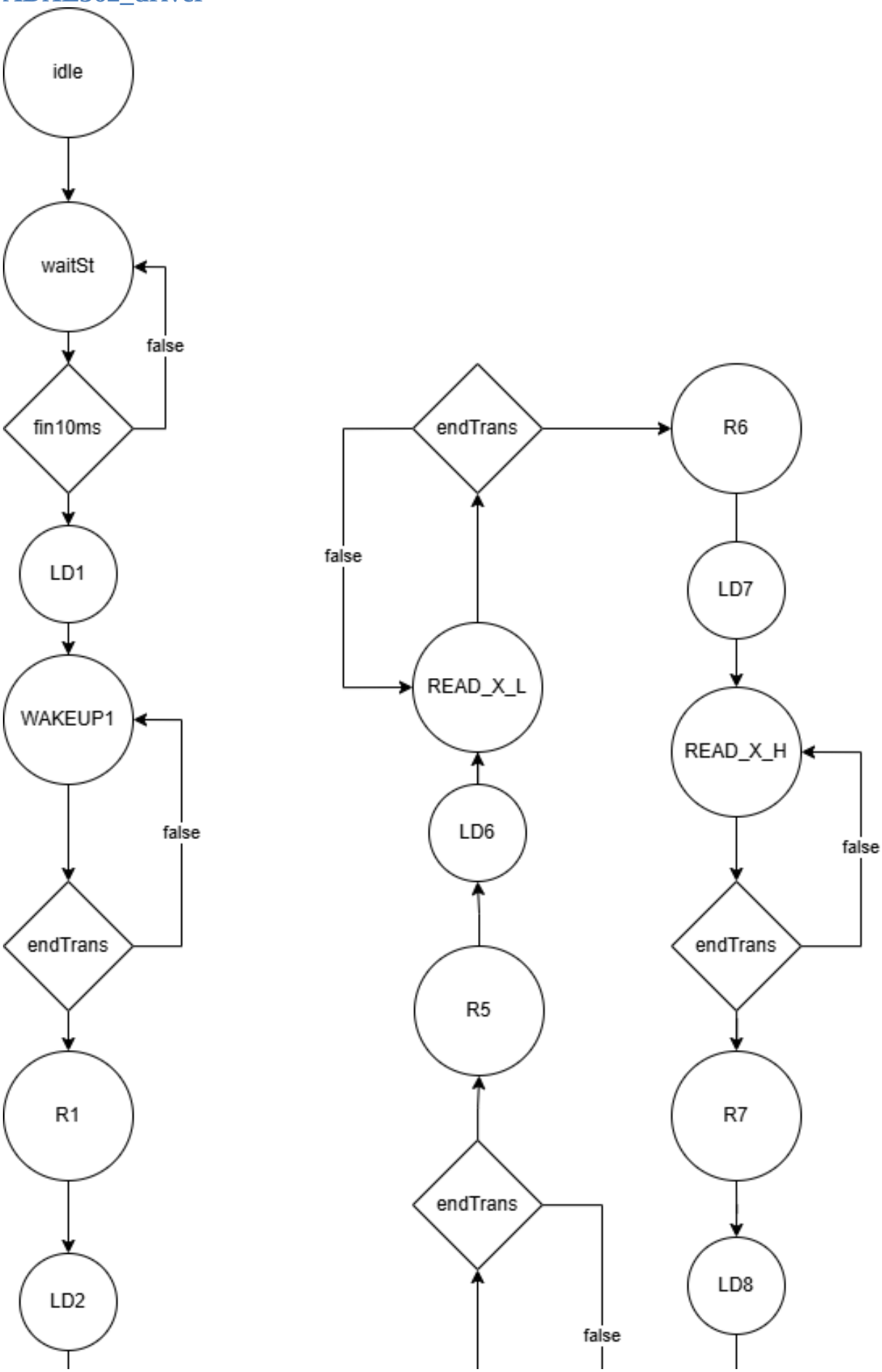

IDLE
Setting Switch
true
settings
false
setBtn
false
true
SetPriority1
set gesture
true
GestSwitch
false
SetPriority2
TiAcSwitch
true
false
set time
set accSens
false
Select mode
Switch mode
true
false
wait for object detection
close state
obj detect btn
true
wait counter
false
2 sec passed
true
confirm object
obj detect btn
btn close
true
close hand man
close hand
false
open state
btn open
true
open hand man
open hand
false
gesture state
btn gesture
true
gesture hand
gesture
false
false
false

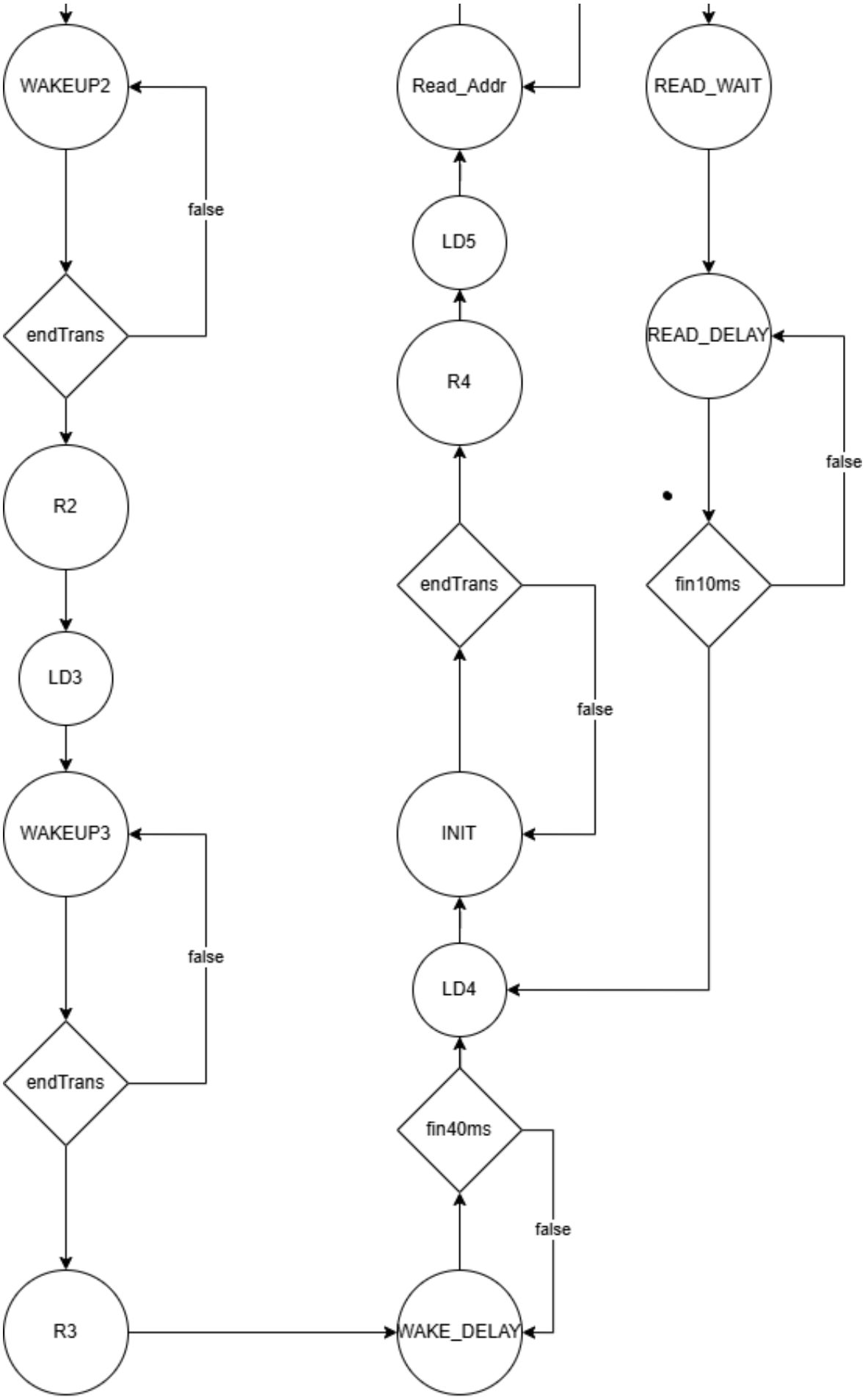

true
close hand
Close Hand signal
decrease counter
x seconds passed
true
true
false
false
wait movement
open hand
movement detected
open hand

CU2

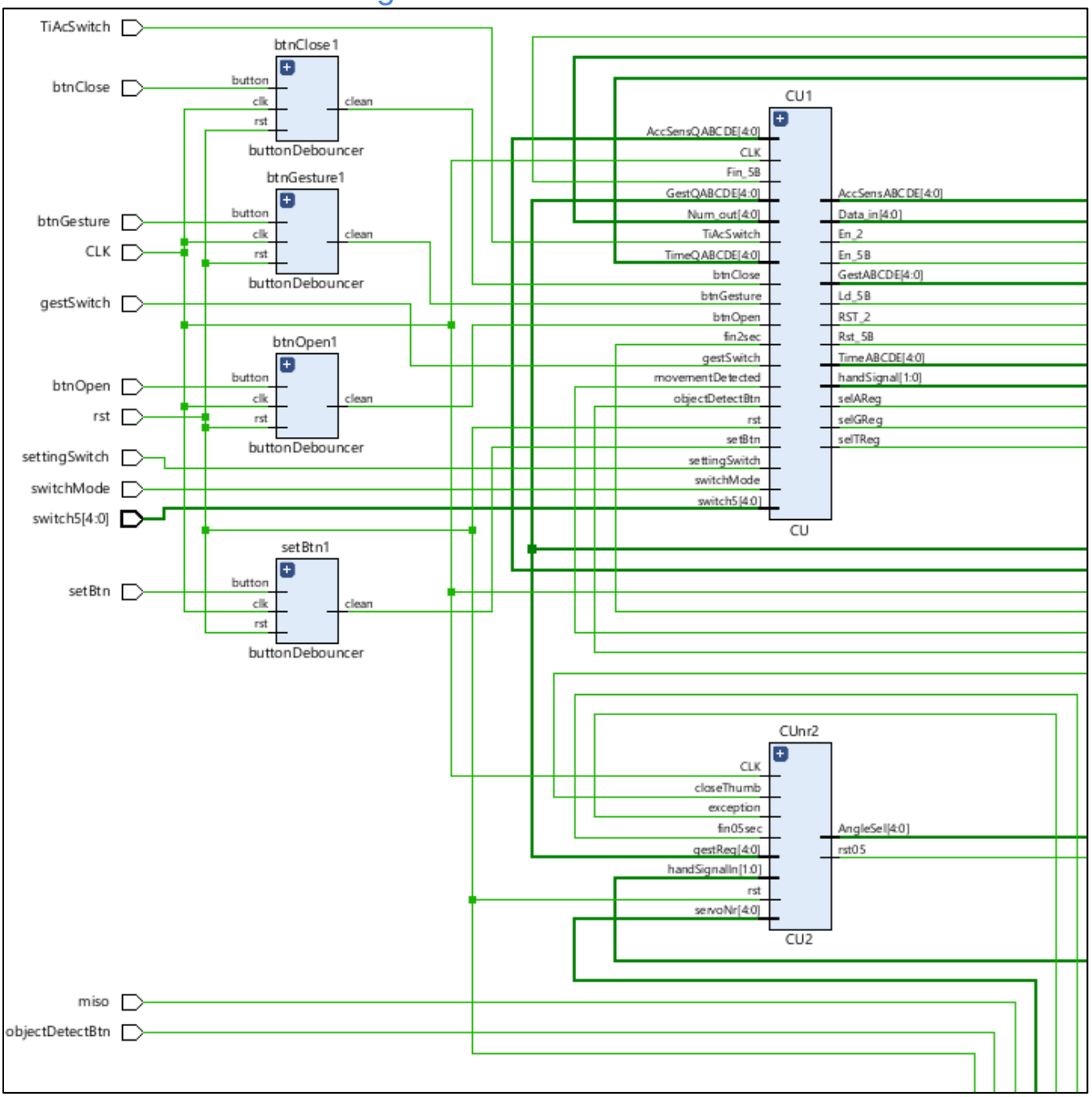

IDLE
Exception
true
false
openClose
closeThumb
true
false
openCloseRest
false
fin05sec
rstClock2
closeThumb
false
fin05sec
openThumb
false
fin05sec
rstClock1
openCLoseRest1
false
fin-5sec
true
openCloseAll
false
fin05sec
true
true

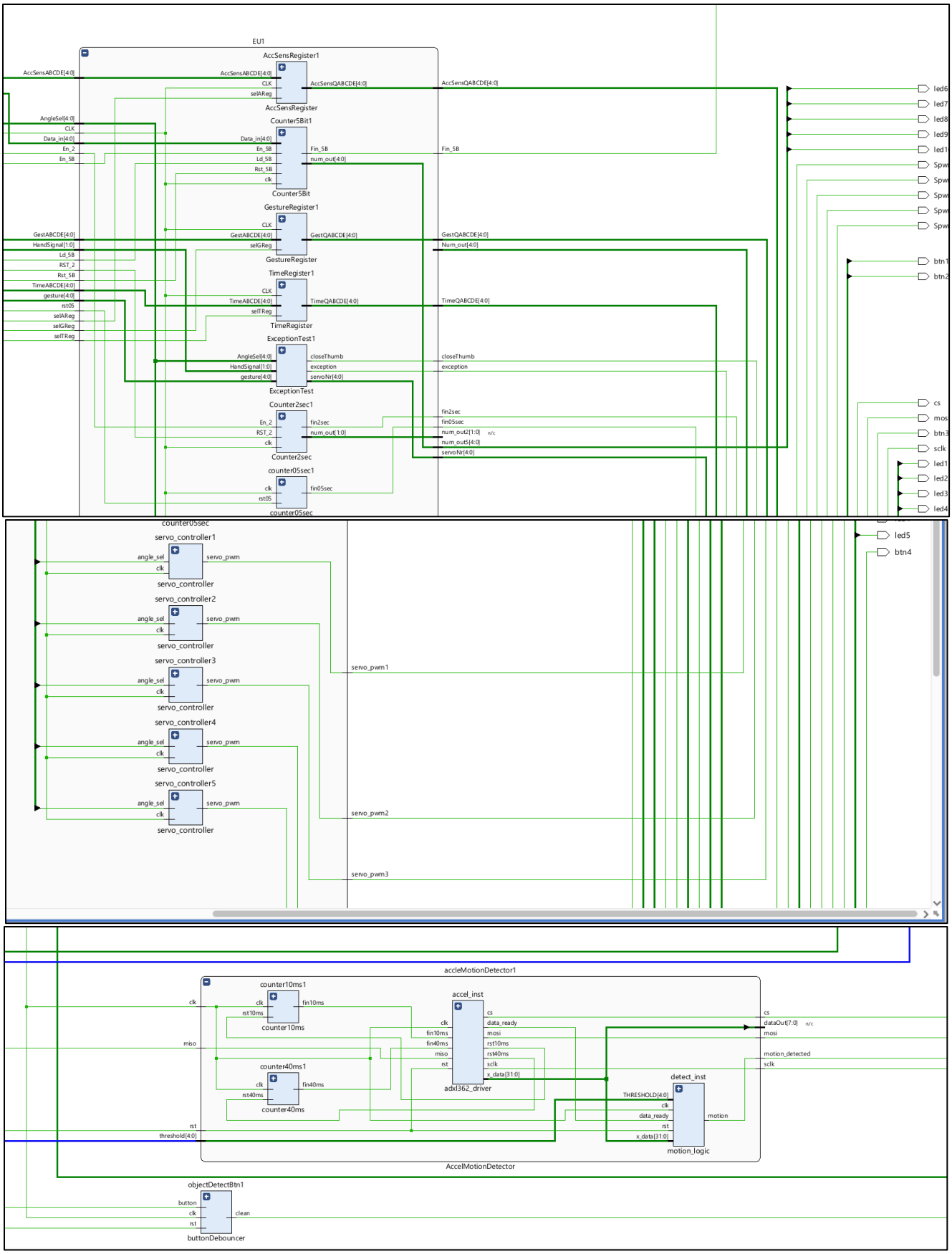
ADXL362_driver




WAKEUP2
false
endTrans
R2
LD3
WAKEUP3
false
endTrans
R3
Read_Addr
LD5
R4
endTrans
false
INIT
LD4
fin40ms
false
WAKE_DELAY
READ_WAIT
READ_DELAY
false
fin10ms

2.4. Detailed diagram




EU1
AccSensRegister1
AccSensRegister
Counter5Bit1
Counter5Bit
GestureRegister1
GestureRegister
TimeRegister1
TimeRegister
ExceptionTest1
ExceptionTest
Counter2sec1
Counter2sec
counter05sec1
counter05sec
servo_controller1
servo_controller2
servo_controller3
servo_controller4
servo_controller5
servo_controller
servo_pwm1
servo_pwm2
servo_pwm3
accleMotionDetector1
counter10ms1
counter10ms
counter40ms1
counter40ms
accel_inst
adxl362_driver
detect_inst
motion_logic
AccelMotionDetector
objectDetectBtn1
buttonDebouncer

3. CHAPTER 3 : USER MANUAL

In order to operate the hand the user can switch between the manual and automatic mode of the with help of a switch. If the switch “switchMode” is switched off the hand will operate in automatic mode.

In automatic mode if the button “objectDetectButton” is pressed for less than 2 seconds nothing will happen, but if it is pressed for more than 2 seconds the hand will close, the user will see the 2 mode leds display 1 in binary and a counter will start to count. As long as no movement in the hand is detected the counter, which value is displayed through the leds on the FPGA, will decrease until 0 then the hand will open, the modeLeds will display 0. But, if movement is detected by the hand, the counter will reset and the hand will remained closed.

In manual mode, meaning the switch “switchMode” is turned on, the user can close, open or make a gesture using the 2 assignated buttons. If a gesture is made the modeLeds will display 2 in binary.

If the user wants to change the gesture, the time of the counter, or the accelerometer sensivity. The user can switch on the settingSwitch entering a setting mode. Using the 5 assignated switches the user can input a 5 bit binary number. If the user wants to introduce the data he needs to press a button “setButton”. To change which data to change the user needs to use the 2 assignated swithces. For example if “gestureSwitch” is switched on the gesture will be changed. For the gesture, every bit in the digit in the number will mean an open or closed finger. For example for the data “10000” introduced in the gesture register, while doing the gesture, the hand will open the first finger and close the others. If the switch “TiAccSwitch” is off, the time data is going to get changed. If turned on, the movement sensitivity is going to get changed.

4. CHAPTER 4 : DESIGN SOLUTIONS JUSTIFICATION

The implementation uses 3 control units. One for the one for the control of the hand, one for the control of the fingers and one for the accelerometer. While the separation between the accelrometer Cu and the other 2 might seem obvious, the separation between the first and second might seem a little bit strang. It could have been done in only one CU by replacing everywhere the handSignal is outputed in the first CU with the content of the second CU. But, this would not take full advantage of the conccurency of an hardware implementation and it would make the testing and development of the hand way harder. By using this design, at any point the data sent to the servos is being computed concurrently meaning the user will have a more seamless experience. For example, if you set a gesture in the settings while performing a gesture, the user will see the data inputed on the fingers as fast as pressing the button, being able to easily set up a gesture or changing fast between them if needed. This is a behaviour that couldn’t have been achieved with only one Control unit.

5. CHAPTER 5 : FURTHER DEVELOPMENT

A way to improve the hand is by using an ultrasonic sensor instead of using a button for detecting an object, improving the user experience in this way.

Another improvement that can be made is by using all the 3 axes that the accelerometer is sensitive to. In the current implementation the hand only reads movement made on the x-axis. By using all the 3 axis, the hand will perform better and it will not open unintentionally while doing a movement that has not been detected on the x-axis

6. CHAPTER 6 : BIBLIOGRAPHY

-https://www.youtube.com/watch?v=7b3YwQWwvXM
-https://vhdlwhiz.com/rc-servo-controller-using-pwm/
-https://www.analog.com/media/en/technical-documentation/data-sheets/adxl362.pdf